src/utils/sPointManager.ts

Below is the complete, strongly typed TypeScript implementation for the S-Point reward and tracking system module.

Service Code Rules Reference

Code	Service Key	S-Points	Sinhala Description
A	red_alert	10.0	රතු එළිය: ආපදා දැනුවත් කිරීමට
B	blood_donation	9.0	එක් ලේ දන්දීමක් වනුවනේ
C	relief_camps	8.0	සහන කඳවුරු ලියාපදිංචිය
D	safe_location	7.0	ස්ථානය සුරකිම: Public
E	quick_service_helper	6.5	ඉක්මන් සේවා සහයවීමට
F	general_service_helper	6.0	සාමාන්‍ය සේවා සහයවීමට
G	found_items	5.5	හමුවනලද
H	job_post	5.0	රැකියා පළකිරීම
I	accommodation_post	4.0	නවාතැන් පළකිරීම
J	emergency_numbers	3.0	හදිසි සේවා අංක
K	professional_service	2.0	වෘත්තිය සේවා පළකිරීම

Implementation Code

```
export type ServiceCode = 'A' | 'B' | 'C' | 'D' | 'E' | 'F' | 'G' |
'H' | 'I' | 'J' | 'K';
```

```
export interface SPointRule {
  code: ServiceCode;
  serviceKey: string;
  points: number;
  descriptionSi: string;
}
export interface UserSPoints {
  totalPoints: number;
  serviceCounts: Record<ServiceCode, number>;
}
export interface User {
  id: string;
  name?: string;
  displayName?: string;
  username?: string;
  sPoints?: Partial<UserSPoints> | number;
}
```

```
export interface LeaderboardEntry {
  rank: number;
  userId: string;
  username: string;
  totalPoints: number;
  summaryCode: string;
  isCurrentUser: boolean;
}
/**
 * Registry of all 11 S-Point earning service rules.
 */
export const S_POINT_RULES: Record<ServiceCode, SPointRule> = {
  A: {
    code: 'A',
    serviceKey: 'red_alert',
    points: 10.0,
    descriptionSi: 'රතු එළිය: ආපදා දැනුවත් කිරීමට',
```

```
},
B: {
  code: 'B',
  serviceKey: 'blood_donation',
  points: 9.0,
  descriptionSi: 'එක ලේ දන්දීමක් වනුවනේ',
},
C: {
  code: 'C',
  serviceKey: 'relief_camps',
  points: 8.0,
  descriptionSi: 'සහන කඳවුරු ලියාපදිංචිය',
},
D: {
  code: 'D',
  serviceKey: 'safe_location',
  points: 7.0,
```

```
  descriptionSi: 'ස්ථානය සුරකිම: Public',
},
E: {
  code: 'E',
  serviceKey: 'quick_service_helper',
  points: 6.5,
  descriptionSi: 'ඉක්මන් සේවා සහයවීමට',
},
F: {
  code: 'F',
  serviceKey: 'general_service_helper',
  points: 6.0,
  descriptionSi: 'සාමාන්ය සේවා සහයවීමට',
},
G: {
  code: 'G',
  serviceKey: 'found_items',
```

```
  points: 5.5,
  descriptionSi: 'හමුවනලද',
},
H: {
  code: 'H',
  serviceKey: 'job_post',
  points: 5.0,
  descriptionSi: 'රැකියා පළකිරීම',
},
I: {
  code: 'I',
  serviceKey: 'accommodation_post',
  points: 4.0,
  descriptionSi: 'නවාතැන් පළකිරීම',
},
J: {
  code: 'J',
```

```
    serviceKey: 'emergency_numbers',
    points: 3.0,
    descriptionSi: 'හදිසි සේවා අංක',
  },
  K: {
    code: 'K',
    serviceKey: 'professional_service',
    points: 2.0,
    descriptionSi: 'වෘත්තිය සේවා පළකිරීම',
  },
};

/**
 * Normalizes user S-Point data, resolving missing fields to standard defaults.
 */
export function getUserSPoints(user: User): UserSPoints {
  const defaultCounts: Record<ServiceCode, number> = {
```

```
    A: 0, B: 0, C: 0, D: 0, E: 0, F: 0, G: 0, H: 0, I: 0, J: 0, K: 0,
  };
  if (!user || !user.sPoints) {
    return { totalPoints: 0, serviceCounts: defaultCounts };
  }
  if (typeof user.sPoints === 'number') {
    return { totalPoints: user.sPoints, serviceCounts: defaultCounts
};
  }
  const serviceCounts = {
    ...defaultCounts,
    ...(user.sPoints.serviceCounts || {}),
  };
  const totalPoints =
    typeof user.sPoints.totalPoints === 'number'
      ? user.sPoints.totalPoints
      : Object.entries(serviceCounts).reduce((acc, [code, count]) => {
          const rule = S_POINT_RULES[code as ServiceCode];
```

```
          return acc + (rule ? rule.points * count : 0);
        }, 0);
  return { totalPoints, serviceCounts };
}
/**
 * Awards S-Points for a specified service activity and returns updated point state.
 */
export function awardSPoints(
  user: User,
  serviceCode: ServiceCode,
  count: number = 1
): UserSPoints {
  const current = getUserSPoints(user);
  const rule = S_POINT_RULES[serviceCode];
  if (!rule) {
    throw new Error(`Invalid service code: ${serviceCode}`);
```

```
  }

  const addedCount = Math.max(0, count);

  const newCounts = {

    ...current.serviceCounts,

    [serviceCode]: (current.serviceCounts[serviceCode] || 0) +
addedCount,

  };

  const newTotal = current.totalPoints + rule.points * addedCount;

  return {

    totalPoints: Number(newTotal.toFixed(2)),

    serviceCounts: newCounts,

  };

}

/**

 * Formats a clean username handle from available user profile
attributes.

 * Example output: "@Akash12483"

 */
```

```
export function generateUsername(user: User): string {
  if (user.username) {
    return user.username.startsWith('@') ? user.username :
`@${user.username}`;
  }
  const rawName = user.displayName || user.name || 'User';
  const cleanName = rawName.replace(/\s+/g,
'').replace(/[^a-zA-Z0-9]/g, '');
  let hash = 0;
  const seedString = user.id || rawName;
  for (let i = 0; i < seedString.length; i++) {
    hash = (hash << 5) - hash + seedString.charCodeAt(i);
    hash |= 0;
  }
  const suffix = Math.abs(hash % 90000) + 10000;
  return `@${cleanName}${suffix}`;
}
/**
```

```
 * Formats service activity into a concise code string.
 * Example output: "H2,E1"
 */
export function formatServicesSummary(
  serviceCounts: Record<ServiceCode, number>
): string {
  if (!serviceCounts) return '';
  return (Object.keys(S_POINT_RULES) as ServiceCode[])
    .filter((code) => serviceCounts[code] && serviceCounts[code] > 0)
    .map((code) => `${code}${serviceCounts[code]}`)
    .join(',');
}
/**
 * Generates a sorted leaderboard array with ranks, formatted handles,
and summaries.
 */
export function getLeaderboardUsers(
  allUsers: User[],
```

```
  currentUser?: User
): LeaderboardEntry[] {
  if (!allUsers || allUsers.length === 0) return [];
  const parsedUsers = allUsers.map((user) => {
    const sPointData = getUserSPoints(user);
    return {
      user,
      totalPoints: sPointData.totalPoints,
      summaryCode: formatServicesSummary(sPointData.serviceCounts),
    };
  });
  parsedUsers.sort((a, b) => b.totalPoints - a.totalPoints);
  return parsedUsers.map((item, index) => ({
    rank: index + 1,
    userId: item.user.id,
    username: generateUsername(item.user),
    totalPoints: item.totalPoints,
```

```
    summaryCode: item.summaryCode,

    isCurrentUser: currentUser ? item.user.id === currentUser.id :
false,

  }));

}
```